

FICHES METHODES PHP

Sommaire

<i>FICHE PHP : AFFICHER LE CONTENU D'UNE REQUETE SUR UNE PAGE</i>	<i>2</i>
<i>FICHE PHP : CONSTRUIRE UNE LISTE A PARTIR D'UNE REQUETE</i>	<i>3</i>
<i>FICHE PHP : AFFICHER LE CONTENU D'UNE REQUETE SUR UNE PAGE APRES LA SAISIE D'UN(DE) CRITERE(S) PAR L'UTILISATEUR</i>	<i>4</i>
<i>FICHE PHP : SAISIR UN ENREGISTREMENT DANS LA BDD A PARTIR D'INTERNET</i>	<i>6</i>
<i>FONCTIONNEMENT MULTI-PAGES EN PHP : COMMENT TRANSMETTRE DES VALEURS D'UNE PAGE A UNE AUTRE</i>	<i>8</i>

FICHE PHP : AFFICHER LE CONTENU D'UNE REQUETE SUR UNE PAGE

METHODE :

- 1°) Etablir une connexion avec la BDD
- 2°) Paramétrer le texte de la requête SQL (Ex \$sql_client = « select * from clients where code_client = '\$zt_code' »)
- 3°) Exécuter la requête dans un recordset
- 4°) Balayer le recordset et afficher le contenu soit :
 - a) en utilisant la procédure « ODBC_all_results » de PHP => 1 seule ligne à écrire mais pas de personnalisation possible
 - b) en parcourant le recordset avec une boucle et en affichant pour chaque ligne les champs voulus => totalement adaptable à chaque cas mais nécessite l'écriture d'un petit algo à chaque fois (*Voir exercice 11_odbc*)
 - c) en utilisant une procédure créée par nos soins (ex : ODBC_recordset_vers_tableau) => inclure dans un premier temps le fichier qui contient la procédure instruction « require » et exécuter ensuite la procédure avec le(s) paramètre(s) correspondants(s). => 2 lignes à écrire, code réutilisable et le niveau de personnalisation dépend du paramétrage de la procédure. (*Voir exercice 13_p2_odbc*)
- 5°) Libérer la mémoire (recordset + connexion)

SYNTAXE :

Etablir une connexion à une BDD	<code>\$nom_connexion = odbc_connect ("nom_lien_odbc", "nom_login", "mot de passe");</code>
Exécuter une requête et renvoyer le résultat dans un recordset	<code>\$nom_recordset = odbc_exec (\$nom_connexion, « texte SQL à exécuter / \$nom_variable_contenant_texte_SQL »);</code>
Afficher le contenu d'un recordset dans un tableau HTML	<code>Echo odbc_result_all(\$nom_recordset)</code>
Balayer toutes les lignes d'un recordset	<code>while (odbc_fetch_row(\$nom_recordset) == TRUE) { } </code>
Afficher la valeur d'un champ d'un recordset pour la ligne en cours	<code>\$nom_variable = odbc_result (\$nom_recordset, "nom_du_champ"); echo \$nom_variable ; </code>
Fermeture du recordset	<code>odbc_free_result(\$result);</code>
Fermeture de la connexion	<code>odbc_close(\$maConnexion);</code>

FICHE PHP : CONSTRUIRE UNE LISTE A PARTIR D'UNE REQUETE

METHODE :

- 1°) Etablir une connexion avec la BDD
- 2°) Paramétrer le texte de la requête SQL
- 3°) Exécuter la requête dans un recordset
- 4°) Créer la liste en HTML et balayer le recordset pour créer pour chaque enregistrement une case (option) de la liste
 - d) en parcourant le recordset avec une boucle et en créant pour chaque ligne du recordset une nouvelle case (option) à la liste avec la valeur à afficher et la valeur à stocker (souvent on affiche le libellé et on stocke le code) => solution entièrement paramétrable mais qui nécessite l'écriture d'un petit algo à chaque fois en mixant le HTML et le PHP. (*Voir exercice 13_p2_odbc*)
 - e) en utilisant une procédure créée par nos soins (ex : ODBC_recordset_vers_liste) => 1 seule ligne à écrire, code réutilisable et le niveau de personnalisation dépend du paramétrage de la procédure (*Voir exercice 14_a_odbc*)
- 5°) Libérer la mémoire (recordset + connexion)

NB : rappel sur les listes

Elles doivent être créées en HTML mais le contenu est souvent paramétré à partir d'une requête, ce qui nécessite de mixer le HTML (pour créer la liste et ses options) et le PHP (pour lire et affiche le contenu de la requête).

SYNTAXE :

Etablir une connexion à une BDD	<code>\$nom_connexion = odbc_connect ("nom_lien_odbc", "nom_login", "mot_de_passe");</code>
Exécuter une requête et renvoyer le résultat dans un recordset	<code>\$nom_recordset = odbc_exec (\$nom_connexion, « texte SQL à exécuter / \$nom_variable_contenant_texte_SQL »);</code>
Liste de valeurs	<pre> <SELECT NAME="nom_liste"> <OPTION VALUE="valeur_à_stocker" > Valeur_à_afficher </OPTION> <OPTION VALUE="valeur_à_stocker" > Valeur_à_afficher </OPTION> ... <OPTION VALUE="valeur_à_stocker" > Valeur_à_afficher </OPTION> </SELECT> </pre>
Balayer toutes les lignes d'un recordset	<pre> while (odbc_fetch_row(\$nom_recordset) == TRUE) { </pre>
Afficher la valeur d'un champ d'un recordset pour la ligne en cours	<pre> \$nom_variable = odbc_result (\$nom_recordset, "nom_du_champ"); echo \$nom_variable ; </pre>
Fermeture du recordset	<code>odbc_free_result(\$result);</code>
Fermeture de la connexion	<code>odbc_close(\$maConnexion);</code>

FICHE PHP : AFFICHER LE CONTENU D'UNE REQUETE SUR UNE PAGE APRES LA SAISIE D'UN(DE) CRITERE(S) PAR L'UTILISATEUR

METHODE GENERALE :

Demander dans un premier temps à l'utilisateur de saisir ses critères, envoyer ensuite les valeurs saisies au serveur qui pourra renvoyer le résultat correspondant après l'interrogation de la BDD. => nécessite 2 pages : 1 pour la saisie des critères, et 1 pour récupérer les valeurs et afficher le résultat correspondant

1°) Construire une première page qui demande à l'utilisateur de saisir son (ses) critère(s) (via une zone de texte, une liste ...). => c'est du HTML pur (sauf si on a une liste paramétrée en fonction d'une requête).

La (les) zone(s) de saisie devra(ont) **obligatoirement** se trouver à l'intérieur d'un formulaire afin que le serveur puisse récupérer la (les) valeur(s) saisie(s).

2°) Transmettre la page au serveur à l'aide d'un bouton de type Submit et indiquer sur quelle page seront affichés les résultats (propriété « action » de la balise <form>)

3°) Construire une deuxième page qui :

- a) récupère la (les) valeur(s) saisie(s). Cette étape est automatique car toute zone de saisie placée à l'intérieur d'un formulaire est transmise à la page suivante sous forme de variable dont le nom est : « \$nom_de_la_zone_saisie »)
- b) établit une connexion avec la BDD
- c) paramètre le texte de la requête SQL en fonction des critères choisis (ie les nom des zones de saisie du formulaire transmis en variables à la 2^{ème} page)
- d) exécute la requête dans un recordset
- e) affiche les résultats

(Voir exercices 13_p1_ODBC et 13_p2_ODBC)

METHODE D'ELABORATION DE LA PAGE 1

1°) Créer un formulaire via les balises <form> et </form>

2°) Spécifier la propriété « action » de la balise <form> qui indique le nom du fichier à charger après le clic du bouton « submit » (ici, le nom de la page 2)

3°) Inclure dans le formulaire les zones de saisie : zones de texte, cases à cocher ...

NB : pour inclure une liste paramétrée à partir d'une requête, consulter la fiche « CONSTRUIRE UNE LISTE A PARTIR D'UNE REQUETE »

Attention : toutes les zones de saisie (listes y compris) doivent obligatoirement se trouver à l'intérieur du formulaire (entre les balises <form> et </form>). Dans le cas contraire, le serveur ne pourrait pas récupérer les valeurs saisies et paramétrer le résultat en conséquence.

4°) Inclure dans le formulaire un bouton de type « submit »

(Voir exercice 13_p1_odbc)

METHODE D'ELABORATION DE LA PAGE 2

1°) Etablir une connexion avec la BDD

2°) Paramétrer le texte de la requête SQL en paramétrant la condition WHERE en fonction des critères saisis en page1. Ex \$sql_client = « select * from clients where code_client = '\$zt_code' »)

Attention ici de respecter la syntaxe SQL :

Nom_du_champ = '\$nom_zone_saisie_page1' pour un champ de format texte

Nom_du_champ = \$nom_zone_saisie_page1 pour un champ de format numérique

Nom_du_champ = # \$nom_zone_saisie_page1 # pour un champ de format date

3°) Exécuter la requête dans un recordset

4°) Balayer le recordset et afficher le contenu

=> cf fiche « AFFICHER LE CONTENU D'UNE REQUETE SUR UNE PAGE »

5°) libération de la mémoire

(Voir exercice 13_p2_odbc)

SYNTAXE :

Bouton submit	<INPUT id=id_bouton type=submit value= « texte à afficher sur le bouton » name=nom_bouton >
Zone de texte	<INPUT id=id_zone_de_texte name=nom_zone_de_texte>
Formulaire	<FORM action = "nom_page_à_afficher_sur_clic_de_submit" METHOD = « get/post »> ... </FORM>
Etablir une connexion à une BDD	\$nom_connexion = odbc_connect ("nom_lien_odbc", "nom_login", "mot_de_passe");
Exécuter une requête et renvoyer le résultat dans un recordset	\$nom_recordset = odbc_exec (\$nom_connexion, « texte SQL à exécuter / \$nom_variable_contenant_texte_SQL »);
Balayer toutes les lignes d'un recordset	while (odbc_fetch_row(\$nom_recordset) == TRUE) { }
Afficher le contenu d'un recordset dans un tableau HTML	Echo odbc_result_all(\$nom_recordset)
Afficher la valeur d'un champ d'un recordset pour la ligne en cours	\$nom_variable = odbc_result (\$nom_recordset, "nom_du_champ"); echo \$nom_variable ;
Fermeture du recordset	odbc_free_result(\$result);
Fermeture de la connexion	odbc_close(\$maConnexion);

FICHE PHP : SAISIR UN ENREGISTREMENT DANS LA BDD A PARTIR D'INTERNET

METHODE GENERALE :

Demander dans un premier temps à l'utilisateur de saisir l'enregistrement (autant de zones de saisie que de champs), envoyer ensuite les valeurs saisies au serveur qui pourra les insérer dans la BDD => nécessite 2 pages : 1 pour la saisie des critères, et 1 pour récupérer les valeurs, les insérer dans la BDD et avertir l'utilisateur que l'insertion s'est bien déroulée.

1°) Construire une première page qui demande à l'utilisateur de saisir les valeurs (via des zones de texte, des listes ...). => c'est du HTML pur (sauf si on a une liste paramétrée en fonction d'une requête).

La (les) zone(s) de saisie devra(ont) **obligatoirement** se trouver à l'intérieur d'un formulaire afin que le serveur puisse récupérer la (les) valeur(s) saisie(s).

2°) Transmettre la page au serveur à l'aide d'un bouton de type Submit et indiquer sur quelle est la page à charger (propriété « action » de la balise <form>)

3°) Construire une deuxième page qui :

- f) récupère la (les) valeur(s) saisie(s). Cette étape est automatique car toute zone de saisie placée à l'intérieur d'un formulaire est transmise à la page suivante sous forme de variable dont le nom est : « \$nom_de_la_zone_saisie »)
- g) établit une connexion avec la BDD
- h) paramètre le texte de la requête SQL (insert into ...) en fonction des valeurs saisies en page1 (ie les nom des zones de saisie du formulaire transmis en variables à la 2^{ème} page)
- i) exécute la requête d'insertion
- j) avertit l'utilisateur que l'insertion s'est bien déroulée.

(Voir exercices 16-a et 16-b)

METHODE D'ELABORATION DE LA PAGE 1

1°) Créer un formulaire via les balises <form> et </form>

2°) Spécifier la propriété « action » de la balise <form> qui indique le nom du fichier à charger après le clic du bouton « submit » (ici, le nom de la page 2)

3°) Inclure dans le formulaire les zones de saisie : zones de texte, cases à cocher ...

NB : pour inclure une liste paramétrée à partir d'une requête, consulter la fiche « CONSTRUIRE UNE LISTE A PARTIR D'UNE REQUETE »

Attention : toutes les zones de saisie (listes y compris) doivent obligatoirement se trouver à l'intérieur du formulaire (entre les balises <form> et </form>). Dans le cas contraire, le serveur ne pourrait pas récupérer les valeurs saisies et insérer les valeurs dans la BDD.

4°) Inclure dans le formulaire un bouton de type « submit »

(Voir exercices 16-a)

METHODE D'ELABORATON DE LA PAGE 2

1°) Etablir une connexion avec la BDD

2°) Paramétrer le texte de la requête SQL en récupérant les valeurs saisies en page1.

Syntaxe : INSERT INTO nom_table (champ1, champ2, ..., champn) VALUES ('\$nom_zone_de_saisie1', '\$nom_zone_de_saisie2', ..., '\$nom_zone_de_saisien')

Attention ici de respecter la syntaxe SQL :

'\$nom_zone_saisie_page1' pour un champ de format texte

\$nom_zone_saisie_page1 pour un champ de format numérique

#\$nom_zone_saisie_page1# pour un champ de format date

3°) Exécuter la requête

4°) Tester si l'insertion s'est bien déroulée et avertir l'utilisateur du résultat

(Voir exercices 16-b)

SYNTAXE :

Zone de texte	<INPUT id=id_zone_de_texte name=nom_zone_de_texte>
Liste de valeurs	<pre> <SELECT NAME ="nom_liste"> <OPTION VALUE ="valeur_à_stocker" > Valeur_à_afficher </OPTION> <OPTION VALUE ="valeur_à_stocker" > Valeur_à_afficher </OPTION> ... <OPTION VALUE ="valeur_à_stocker" > Valeur_à_afficher </OPTION> </SELECT> </pre>
Formulaire	<pre> <FORM action = "nom_page_à_afficher_sur_clic_de_submit" METHOD = « get/post »> ... </FORM> </pre>
Bouton submit	<INPUT id=id_bouton type=submit value= « texte à afficher sur le bouton » name=nom_bouton >
Etablir une connexion à une BDD	\$nom_connexion = odbc_connect ("nom_lien_odbc", "nom_login", "mot_de_passe");
Exécuter une requête et renvoyer le résultat dans un recordset	\$nom_recordset = odbc_exec (\$nom_connexion, « texte SQL à exécuter / \$nom_variable_contenant_texte_SQL »);
Afficher le contenu d'un recordset dans un tableau HTML	Echo odbc_result_all(\$nom_recordset)
Balayer toutes les lignes d'un recordset	<pre> while (odbc_fetch_row(\$nom_recordset) == TRUE) { } </pre>
Afficher la valeur d'un champ d'un recordset pour la ligne en cours	<pre> \$nom_variable = odbc_result (\$nom_recordset, "nom_du_champ"); echo \$nom_variable ; </pre>
Fermeture du recordset	odbc_free_result(\$result);
Fermeture de la connexion	odbc_close(\$maConnexion);

FONCTIONNEMENT MULTI-PAGES EN PHP : COMMENT TRANSMETTRE DES VALEURS D'UNE PAGE A UNE AUTRE

Une page Php contient presque toujours un mix d'HTML et de Php (code exécuté côté serveur).

Le serveur interprète la page en la lisant du haut vers le bas et remplace toutes les parties php par le résultat en HTML.

Une page Php fonctionne toujours de façon autonome. On ne peut donc pas avoir accès sur une page à des variables ou à des zones de saisie qui ont été déclarées ou font partie d'une autre page. Dès que l'on passe à une page suivante, la page est en effet entièrement vidée de la mémoire.

Or, il est souvent, il est nécessaire de transmettre des valeurs d'une page à une autre. Pour cela, on dispose de 3 solutions :

1°) On utilise une balise d'ancre placée sur la 1^{ère} page qui va appeler la 2^{ème} page en lui transmettant 1 ou plusieurs valeurs.

Syntaxe :

`<A href="nom_du_fichier?nom_variable = valeur à transmettre"> texte à afficher </A>`

Exemple :

`<A href="page2.php?code= 10"> Aller à la page 2 en transmettant le code N° 10 </A>`

2°) On récupère en page 2 les valeurs saisies dans les contrôles de la 1^{ère} page (zones de textes, listes ...) en utilisant sur la 1^{ère} page un bouton submit et un formulaire (qui englobe tous les contrôles).

Principe : lorsque l'on clique sur le bouton submit, la page précisée dans la propriété « action » du formulaire est chargée et une liste de variables lui est transmise automatiquement. Ces variables portent le nom des contrôles inclus dans le formulaire précédé du signe \$.

NB : le formulaire ainsi que le bouton submit sont à déclarer en HTML.

Syntaxe :

`<FORM action = "nom_page_à_afficher_sur_clic_de_submit" METHOD = « get/post »>`

`<INPUT TYPE = "submit" name = bt_submit value="soumettre" >
</FORM>`

Exemple :

Sur la page 1 :

`<FORM action = "page2.php" METHOD = post >
Saisir votre nom <INPUT id=zt_nom name=zt_nom>
Saisir votre code <INPUT id=zt_code name=zt_code>`


```
<INPUT TYPE = "submit" name = bt_submit value="soumettre" >
</FORM>
```

Sur clic du bouton submit, la page 2 va être chargée et les variables \$zt_nom et \$zt_code auront été transmises et initialisées avec les valeurs saisies en page1.

3°) On crée une session dans la quelle on va stocker des variables.

Tant que la session n'est pas fermée (automatiquement au bout de 20 minutes sans action de l'utilisateur ou explicitement avec l'instruction session_destroy()), on peut avoir accès aux variables session de n'importe quelle page.

NB : les sessions utilisant des ressources mémoires, on préférera dans la mesure du possible utiliser les solutions 1 et 2.

Syntaxe :

Pour ou ouvrir une session ou accéder à la session : `Session_start()`

Attention : cette instruction doit être placée en toute première ligne de la page (avant même le HTML)

Créer une variable session : `Session_register("nom_variable")`

NB : le nom de la variable n'est pas précédé du \$

Pour utiliser une variable session : `$nomvariable_session`

Exemple :

Sur la page 1 :

```
Session_start() ;           //ouverture de session
Session_register("code");   //création de la variable session « code »
$code = 10 ;                 //affectation de la valeur 10 à la variable session « code »
```

Sur la page 2 :

```
Session_start() ;           //accès à la session en cours
Echo $code ;                 //affichage de la variable session « code »
```

(Voir exercice 15)